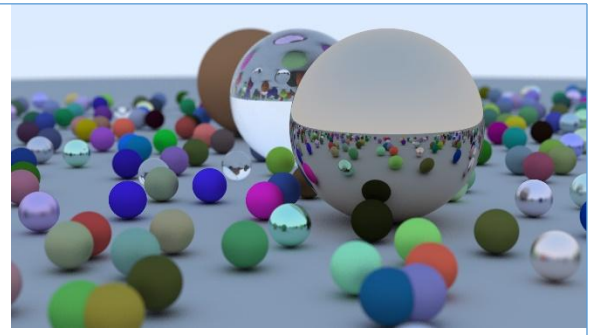
Comp4422



Computer Graphics

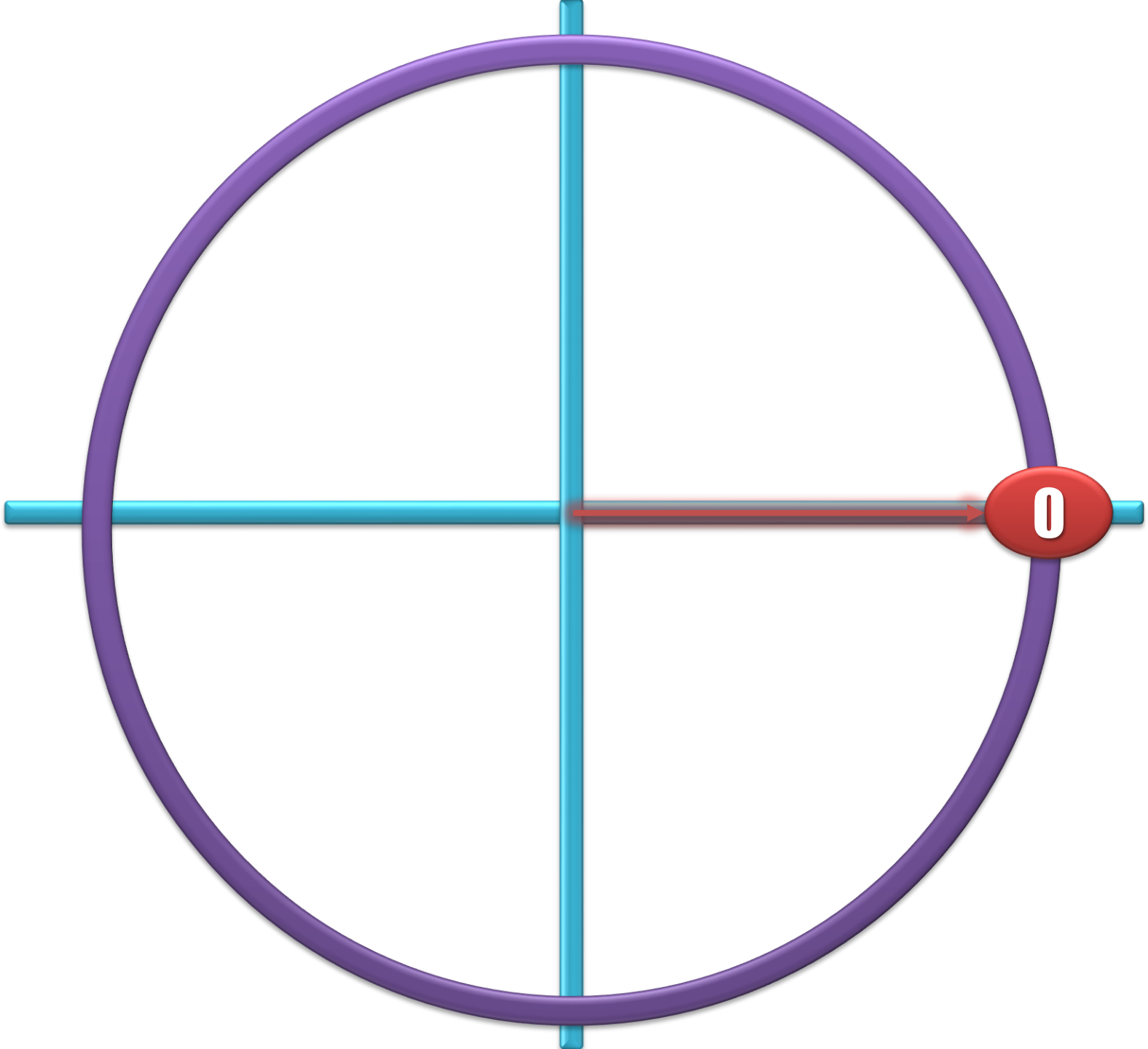
Lecture 09:

Texture Mapping



What you will learn…

- Why textures in computer graphics
- How to generate textures
- How to store textures
- How to apply textures
- GPU-based texture mapping
- Combine several textures


0

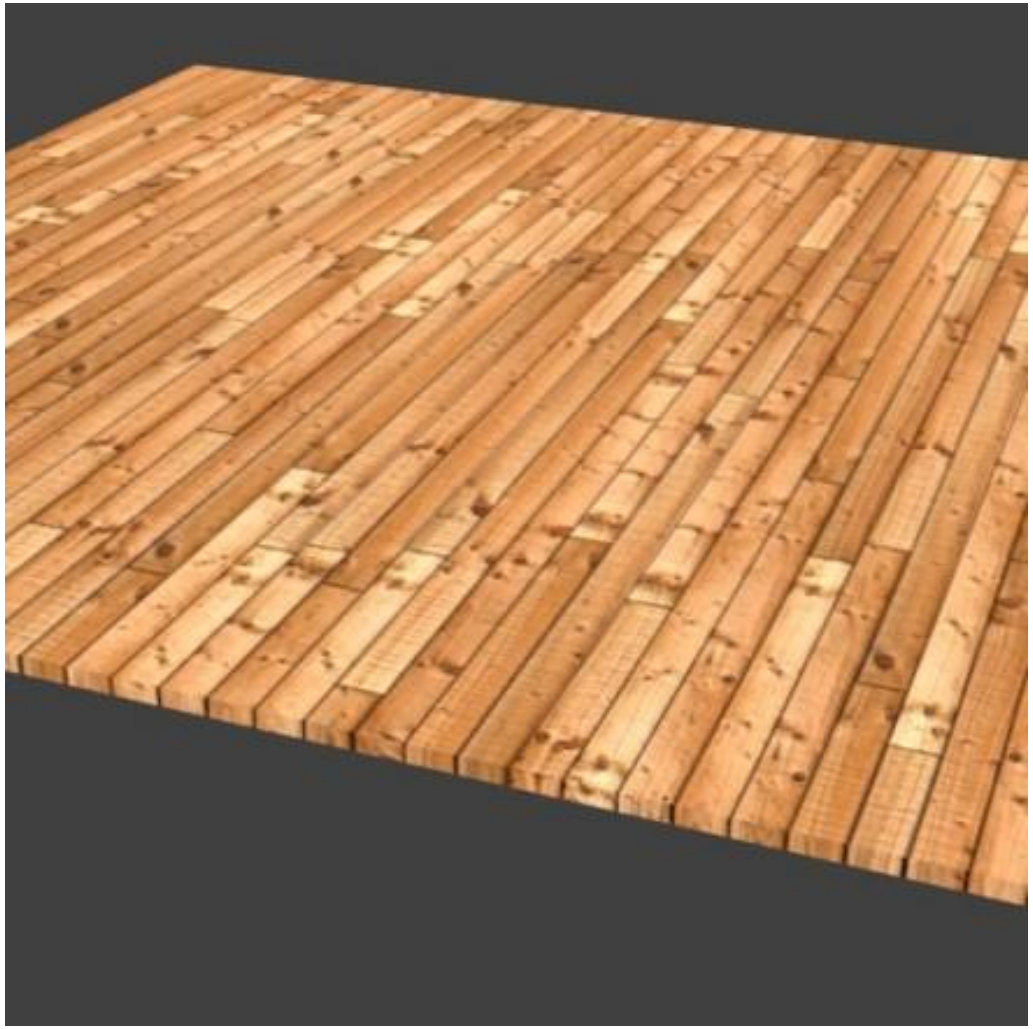
Why textures?

- Textures represent variations in reflected spectral light over a surface
- Textures are color patterns that represent
  - complex surface geometry
  - complex surface composition
- Textures add realism to simple geometry

Example 1

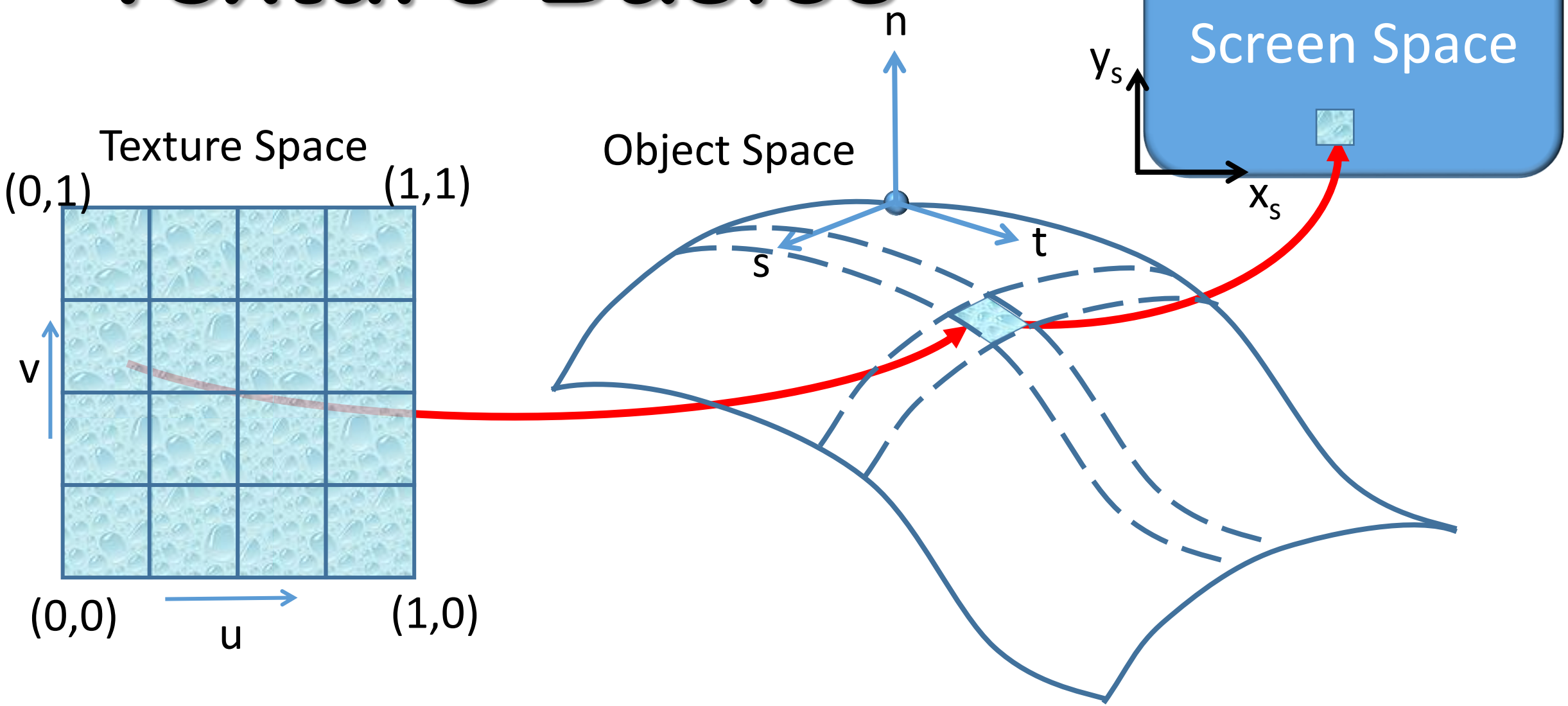


Example 2

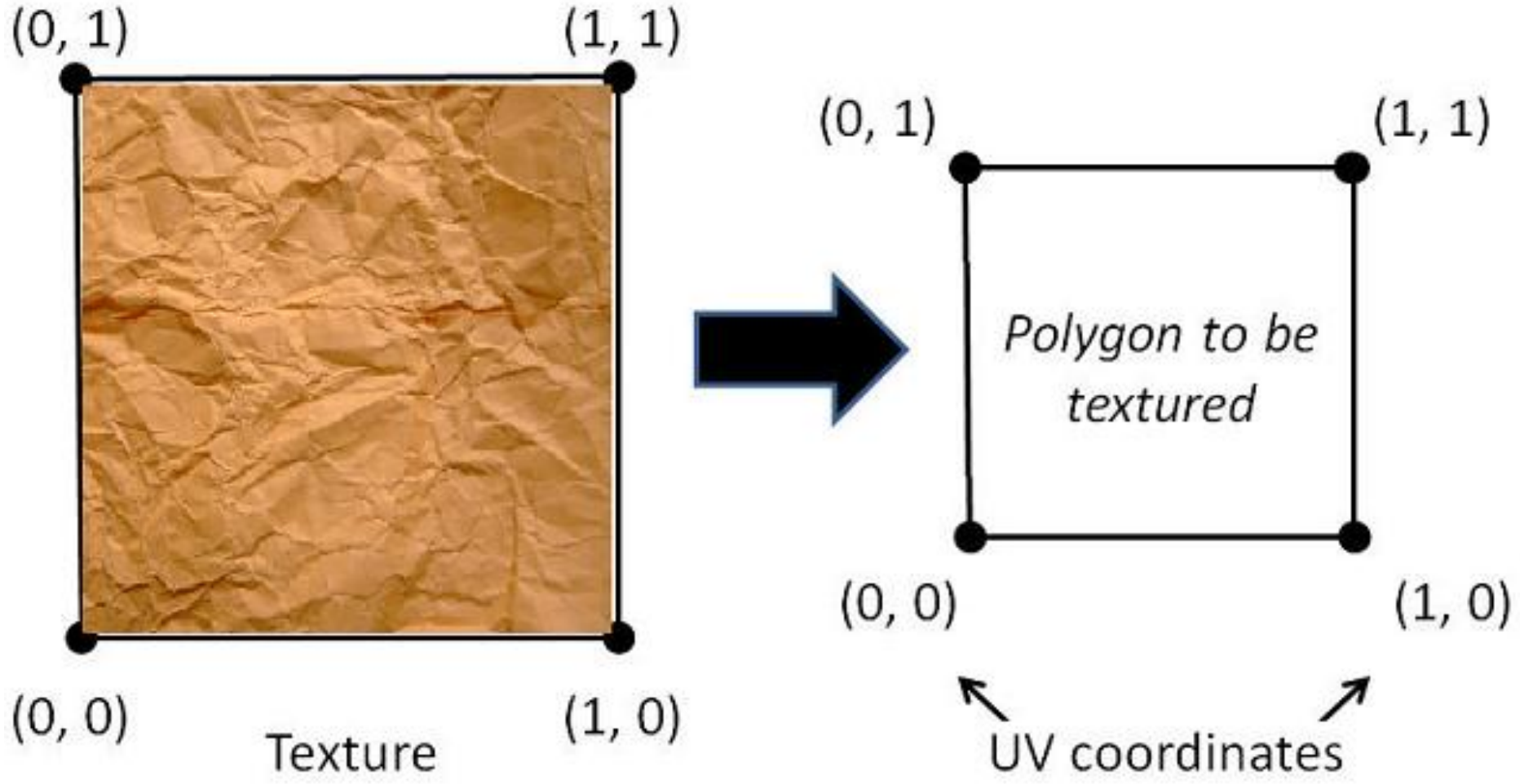




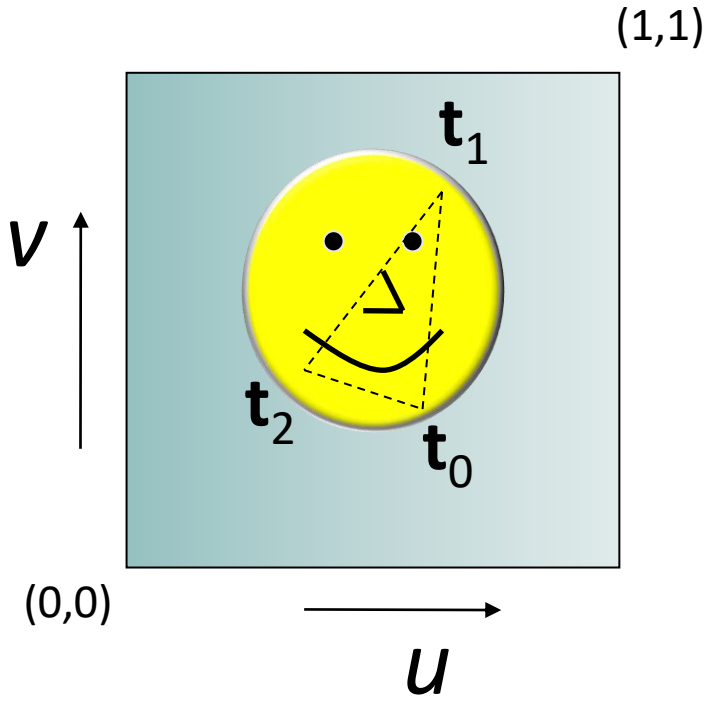
Texture Basics

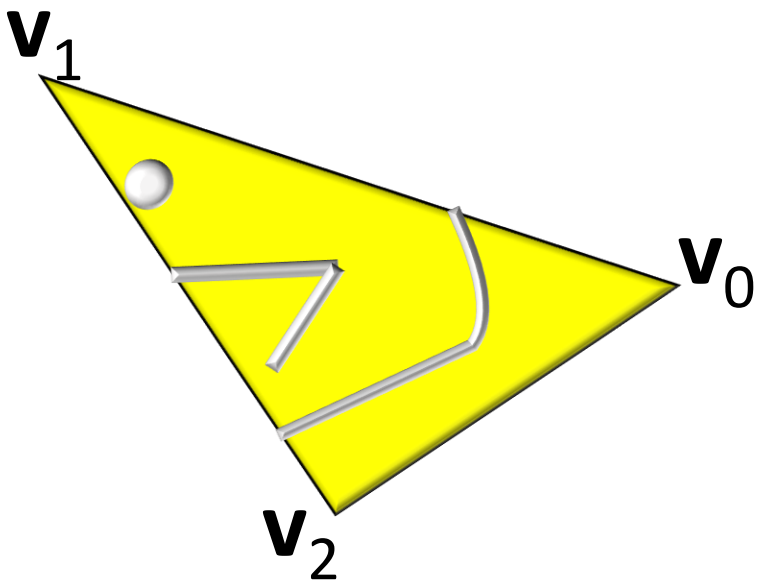


Texture Generation



For a triangle

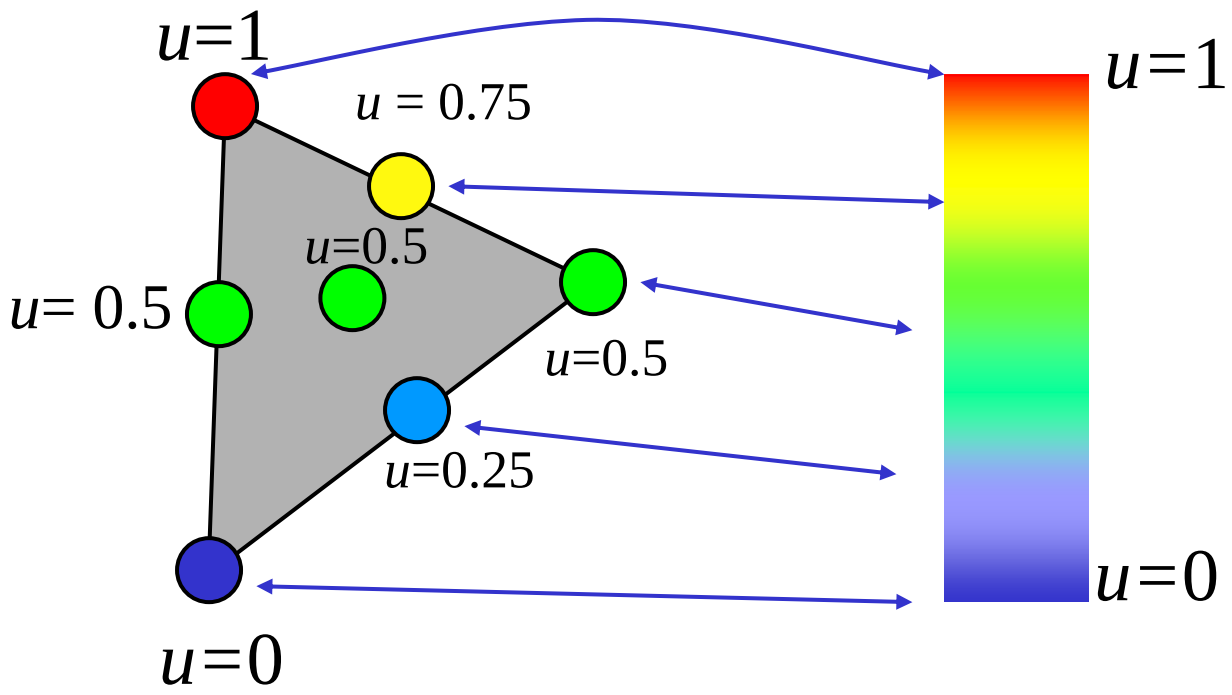




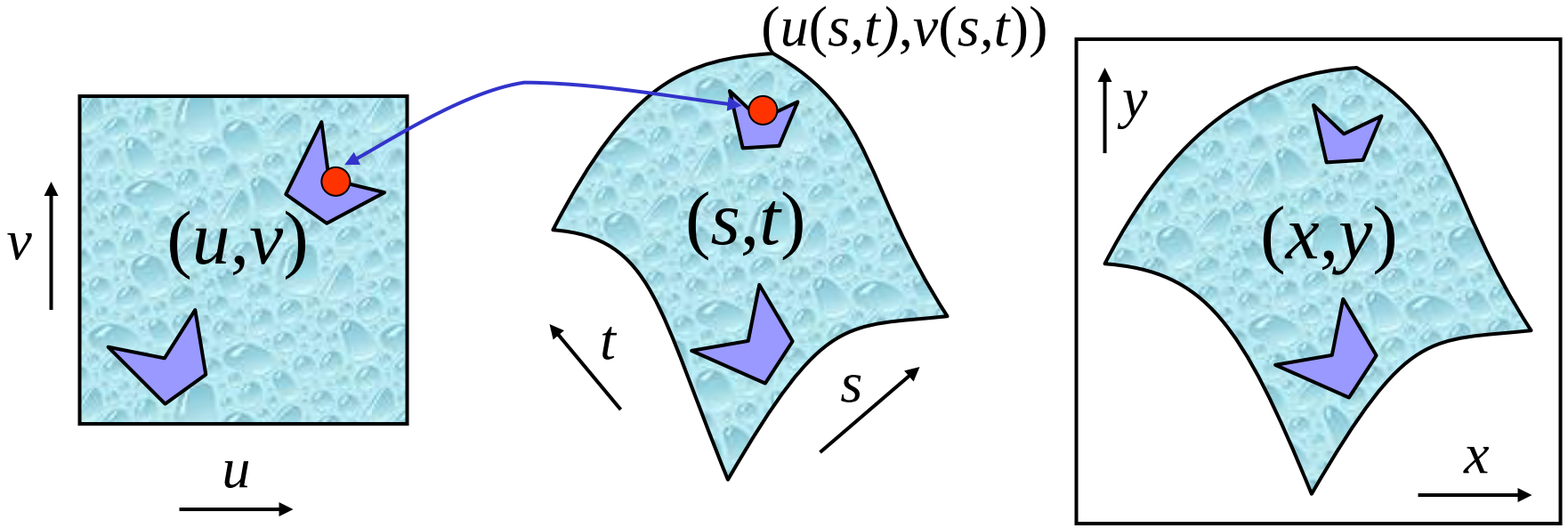
Texture Mapping

Simple 1D Interpolation

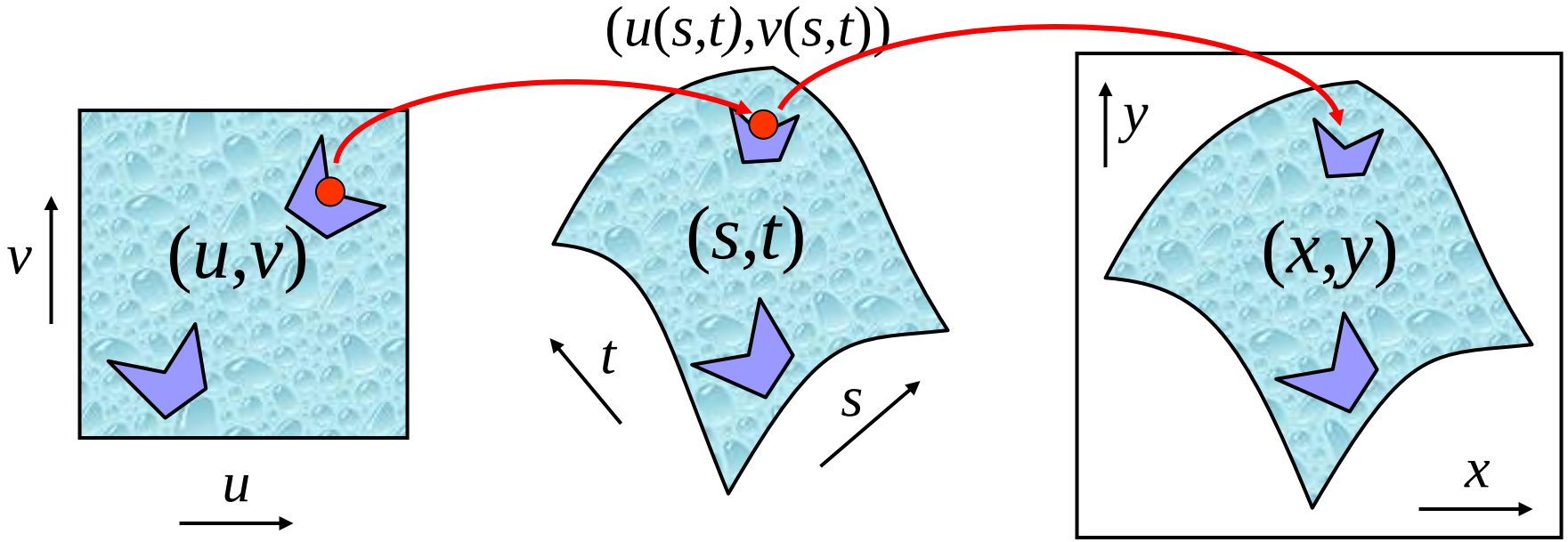
- Coloring a triangle



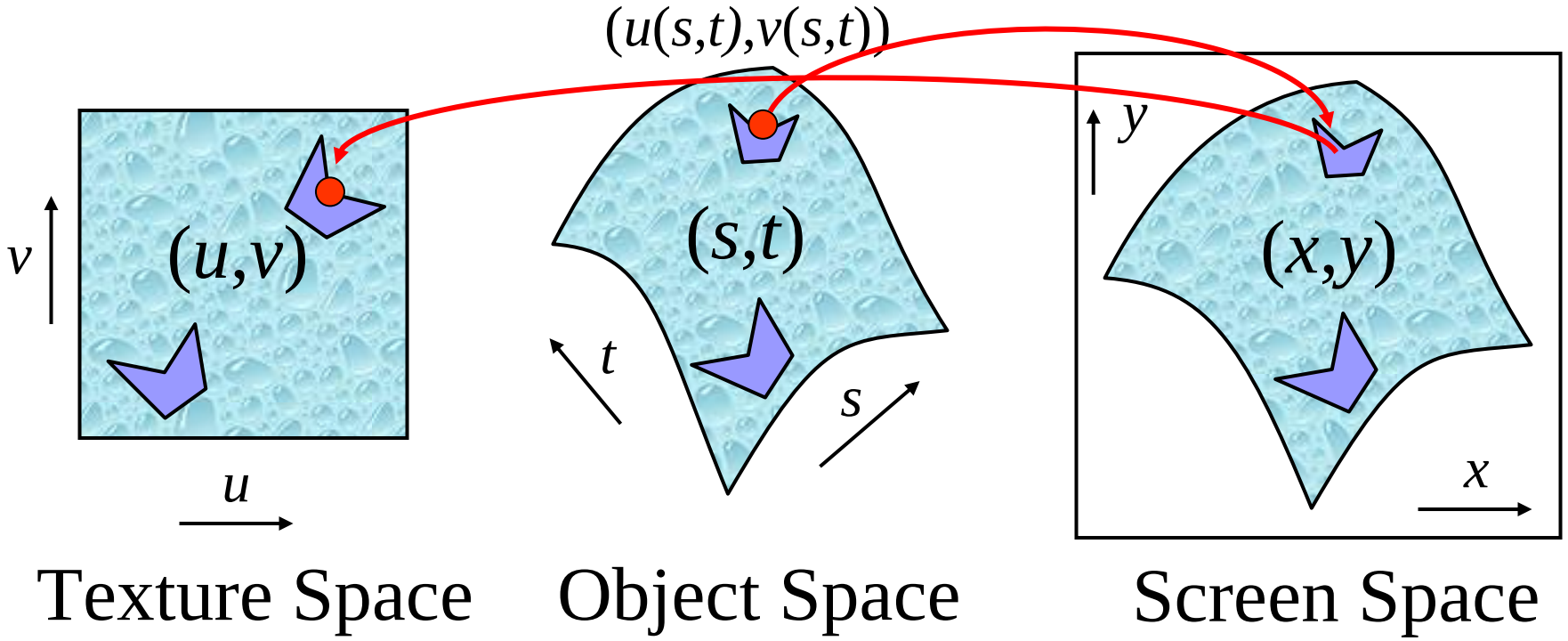
Texturing Surfaces



Forward Texture Map



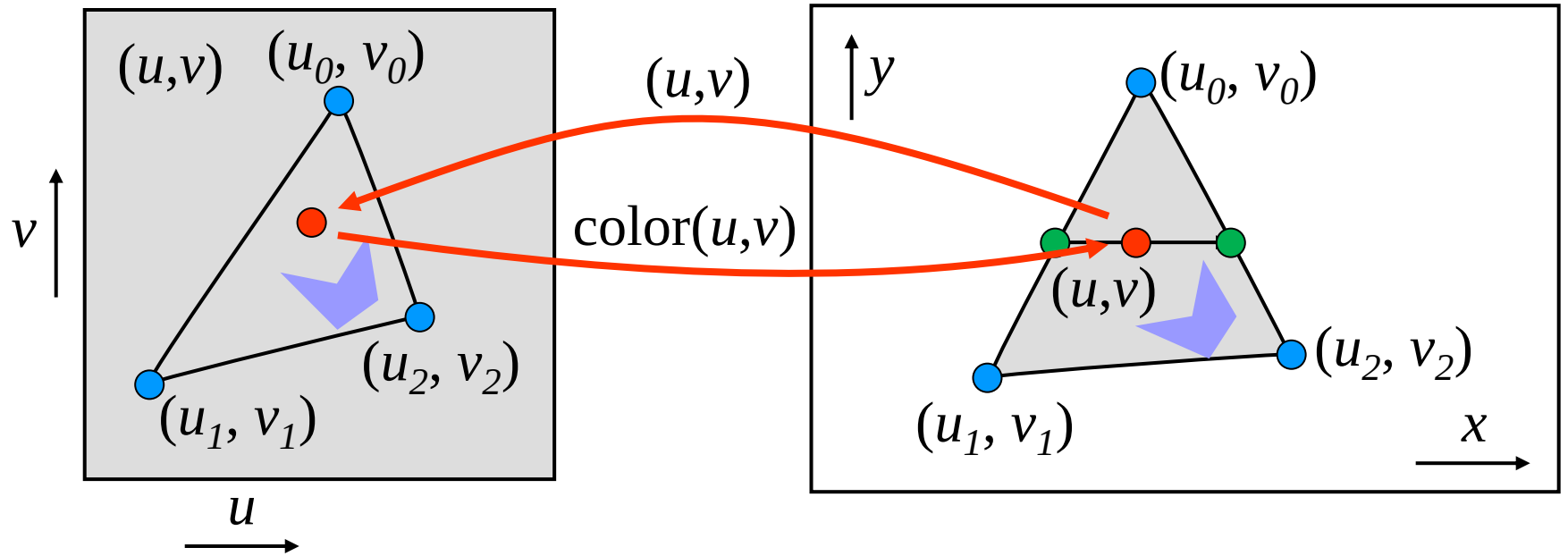
Backward Texture Map



Texture Mapping Triangles

Backward Texture Map

- Each vertex: (x,y,z,u,v)



Backward Texture Map

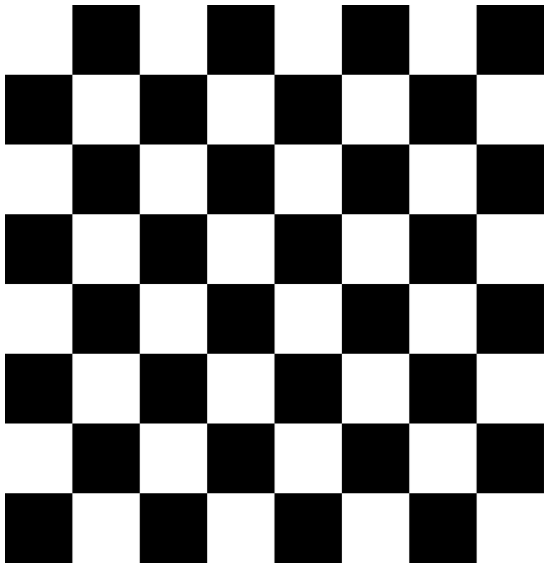
- At each vertex specify
  - Position Coordinates (x,y,z)
  - Texture Coordinates (u,v)
- Look up the color/texture at scan conversion/rasterization by interpolating the raster position into texture image space

3D Texture Map

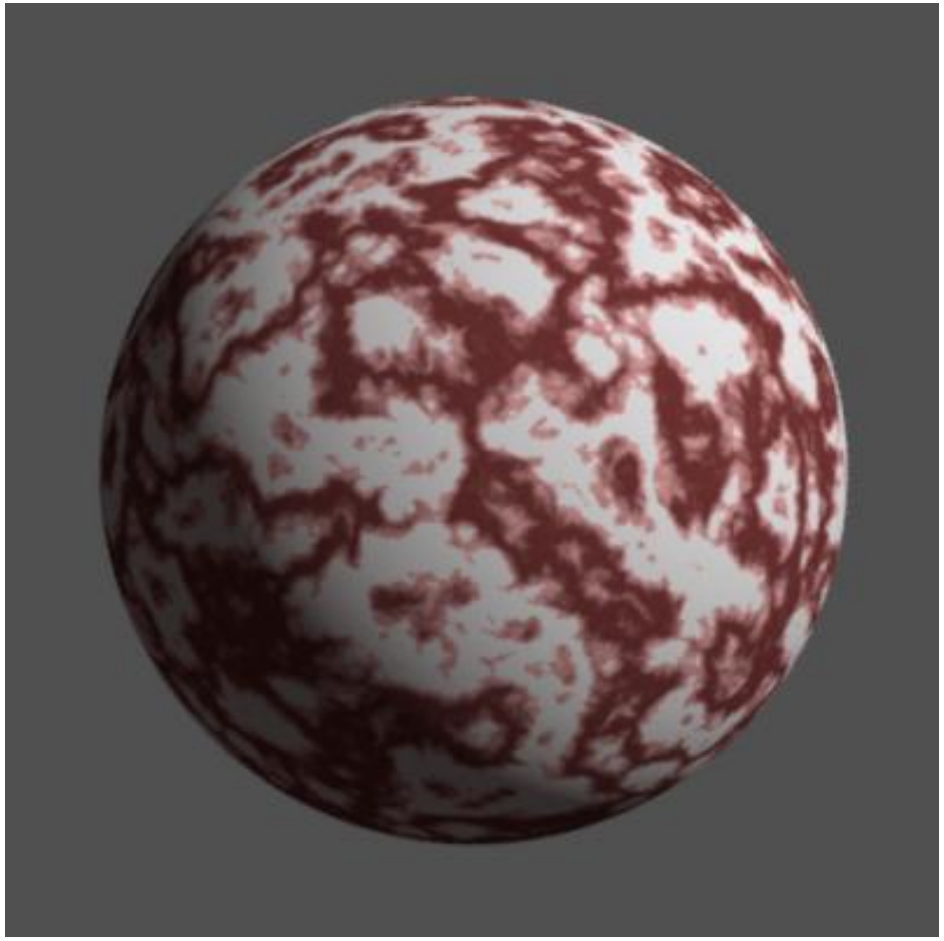
- The texture map is a volume
  - a stack of bitmaps - memory intensive
- Per vertex (*s*,*t*,*r*) coordinates
- Applications:
  - medical 3D images (plane through scan)
  - solid materials (wood)

Procedural Texture Map

- Instead of using a 1-, 2-, or 3-D image:
  - Define function that returns color dependent on value of s, (s,t) or (s,t,r)
- Simple example: a chessboard
- Advanced: wood, marble, etc.

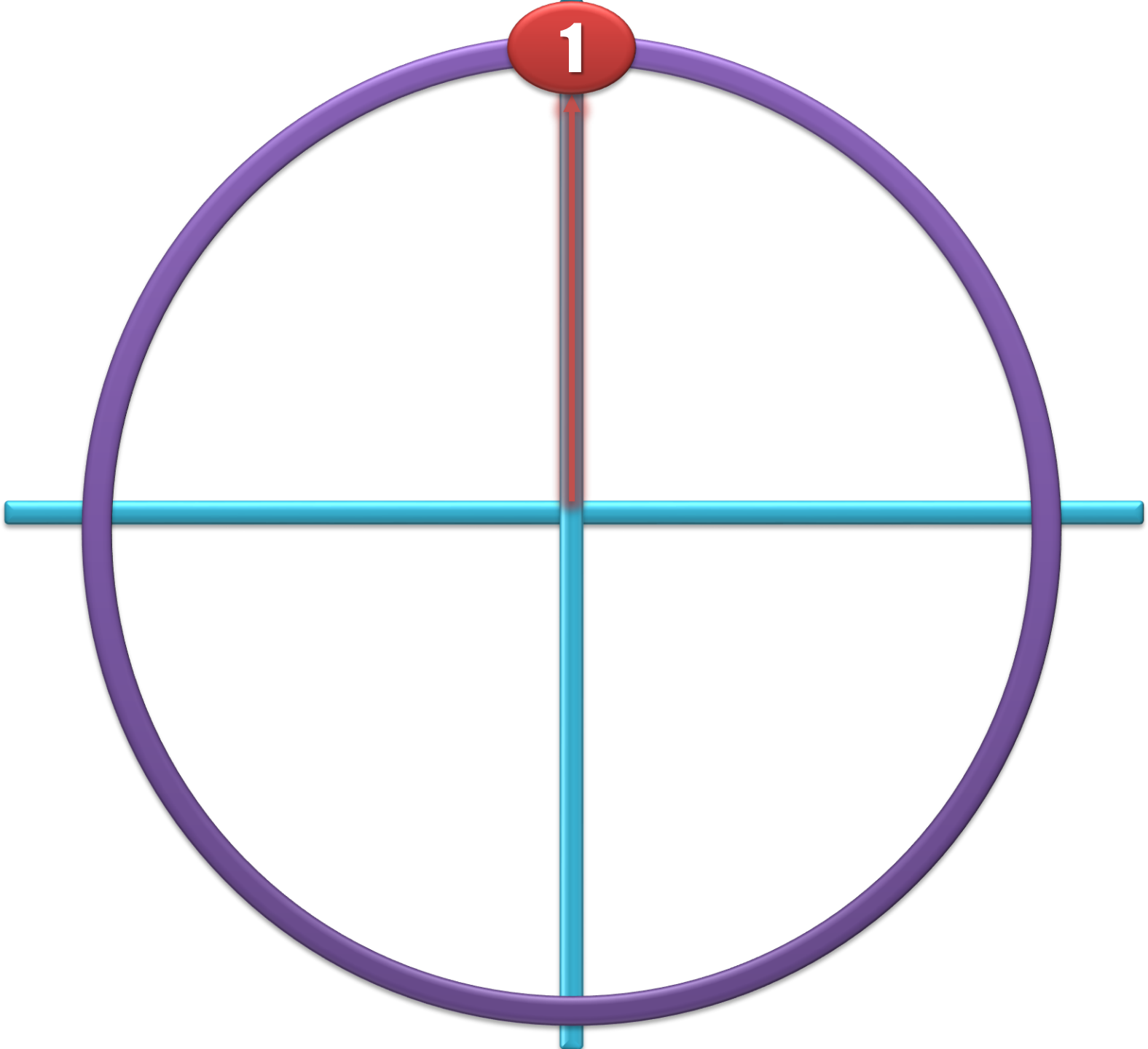


Example: Perlin Texture





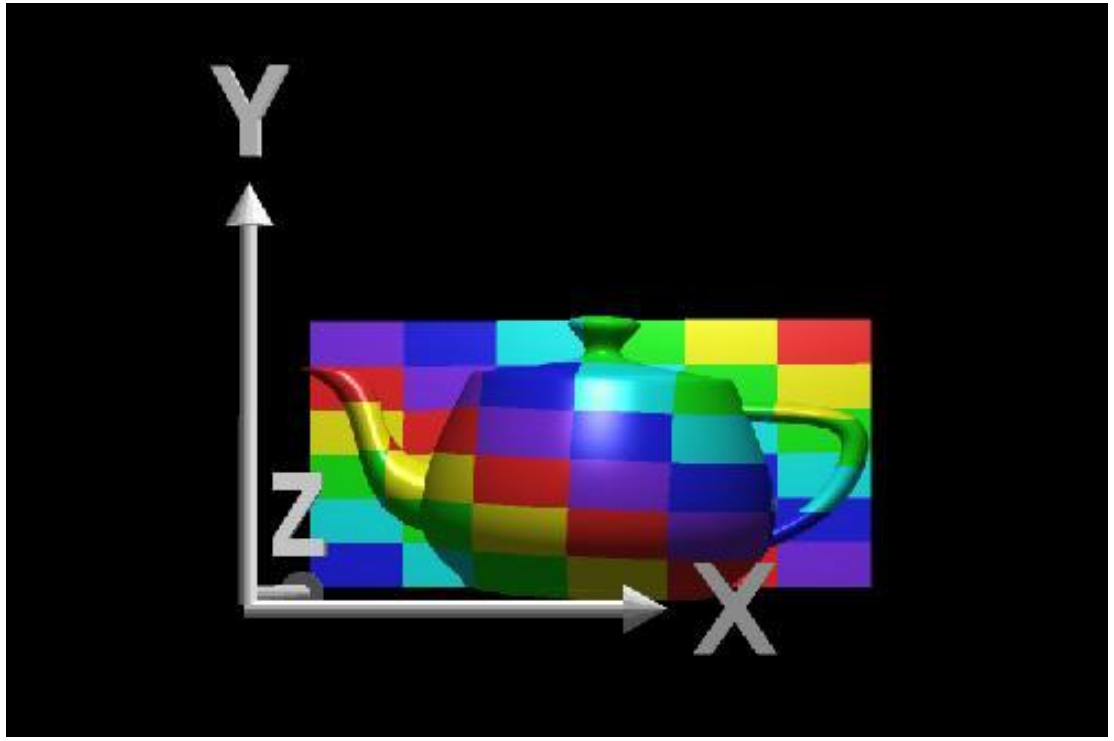
Images by Ken Buckner


1

Advanced Mapping

Intermediate surfaces

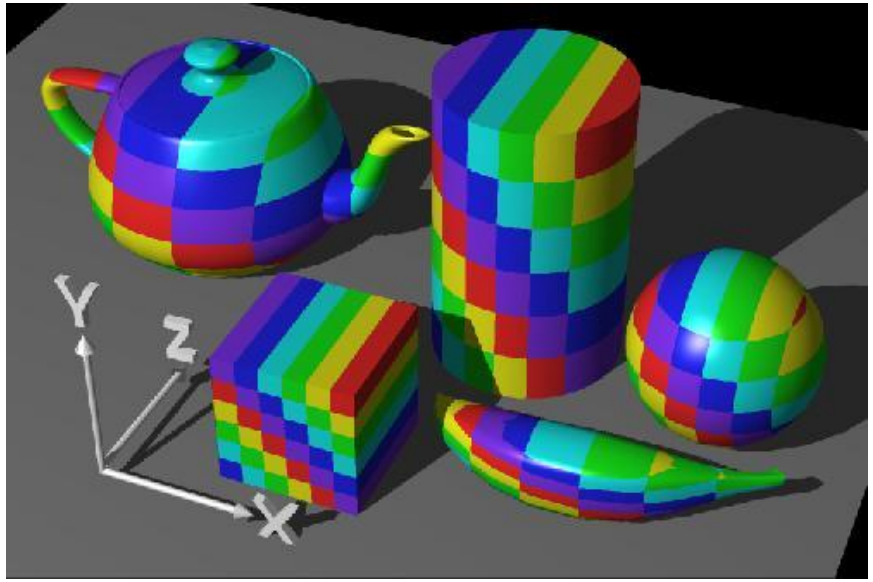
- Plane
- Cylinder
- Sphere
- Cube



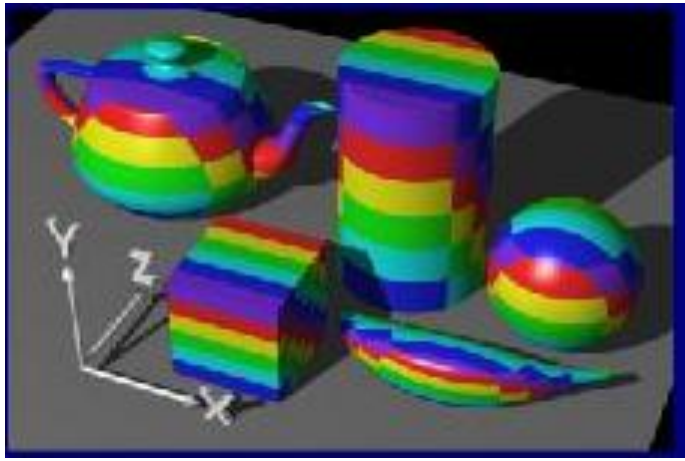
Planar projector

Planar Orthographic

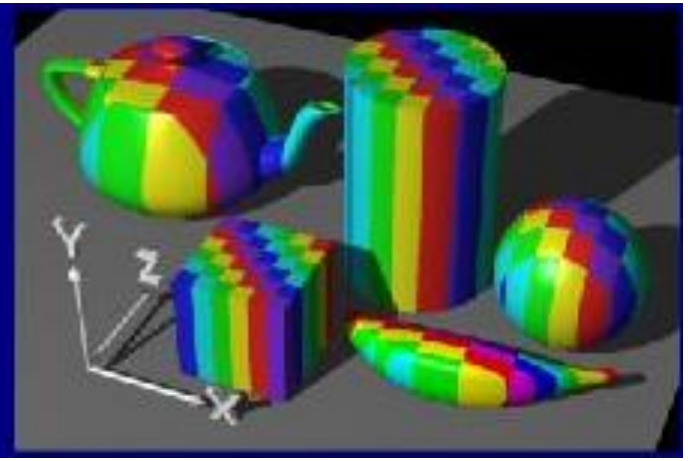
Orthographic projection
onto XY plane:
$u = x,\ v = y$



...onto YZ plane



...onto XZ plane



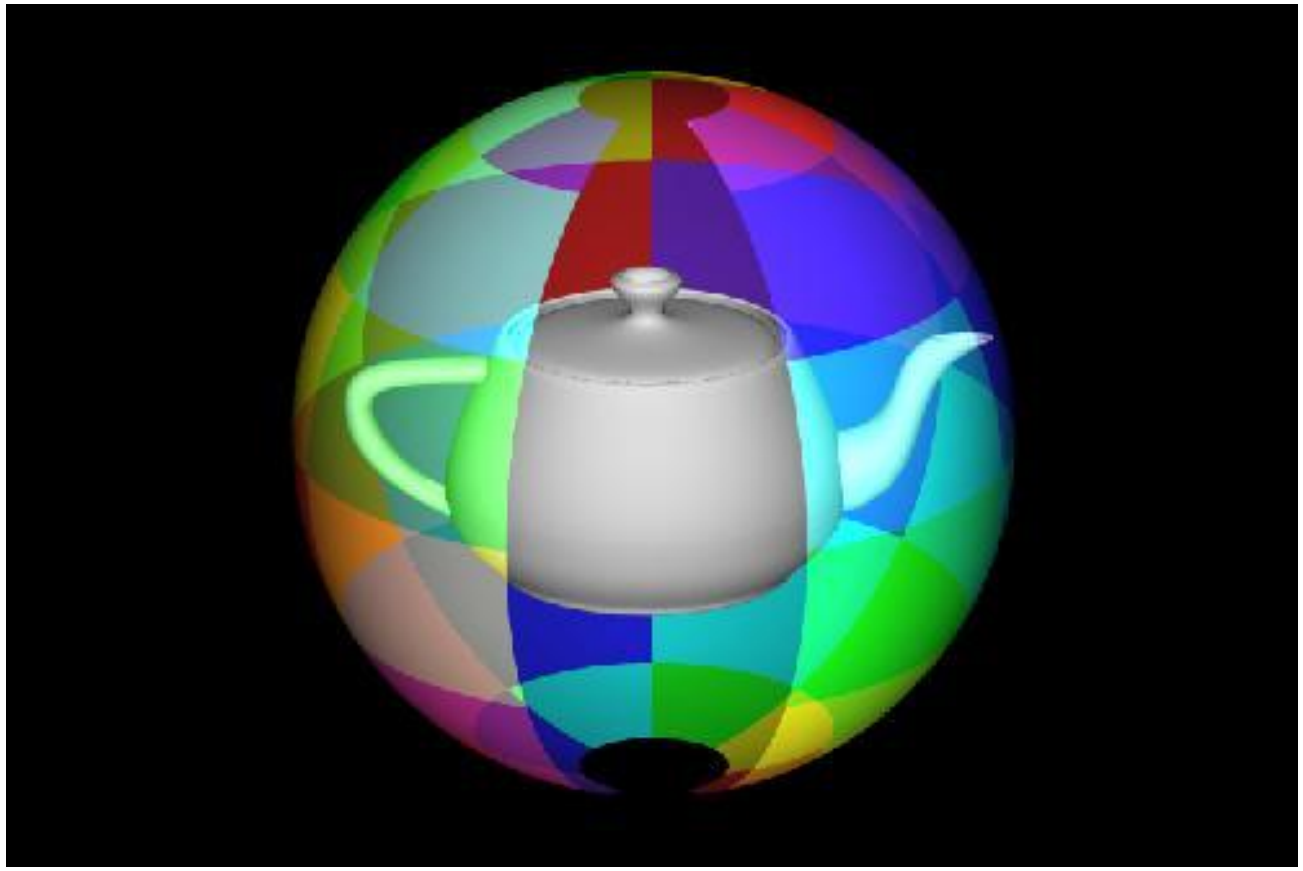
Cylindrical

- Convert rectangular coordinates (x, y, z) to cylindrical (r, ϕ, h),
- Use only (h, ϕ) to index texture image



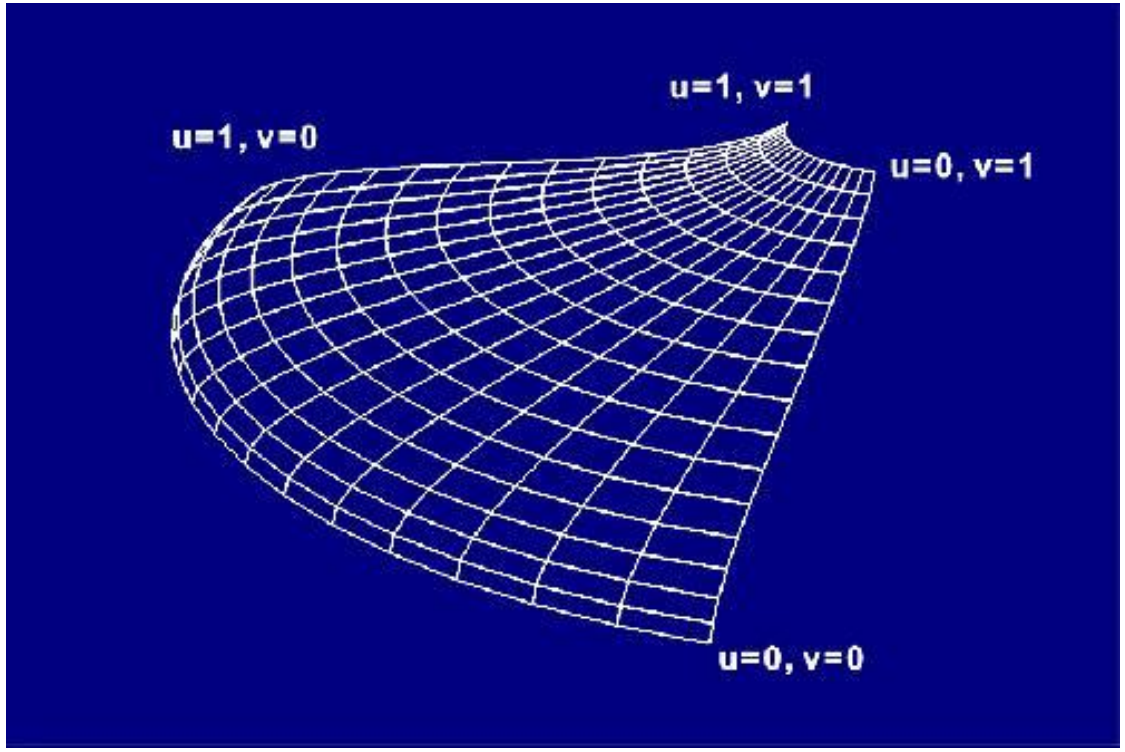
Spherical

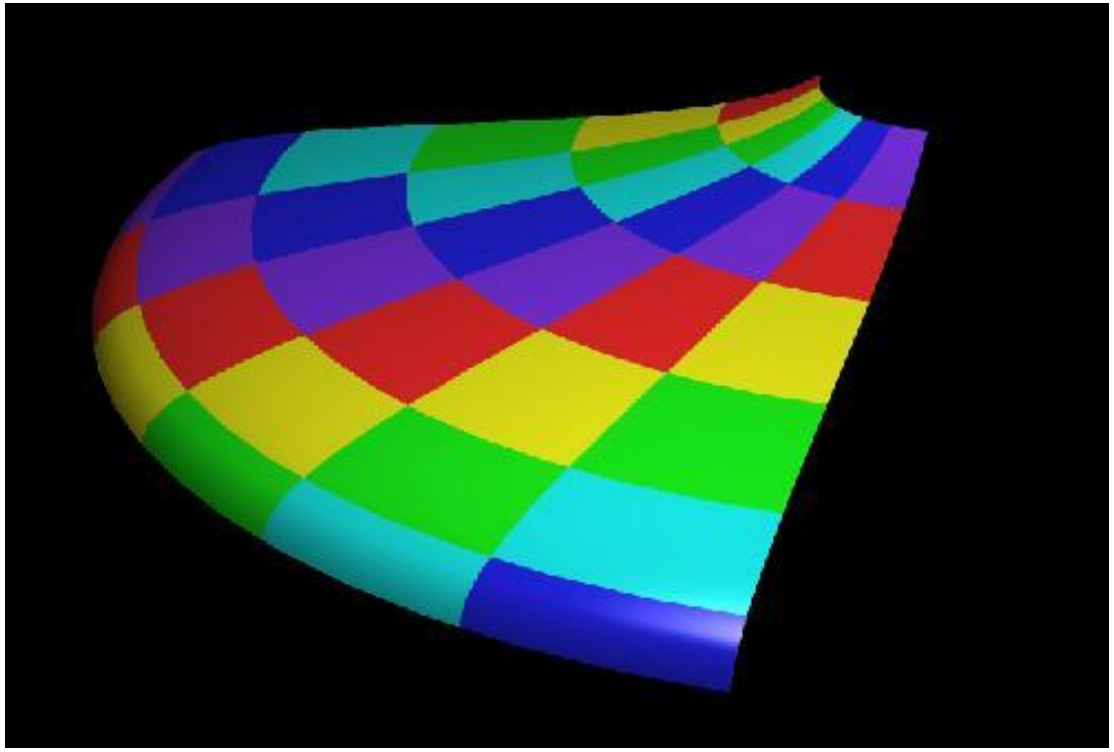
- Convert rectangular coordinates (x, y, z) to spherical (θ, ϕ)



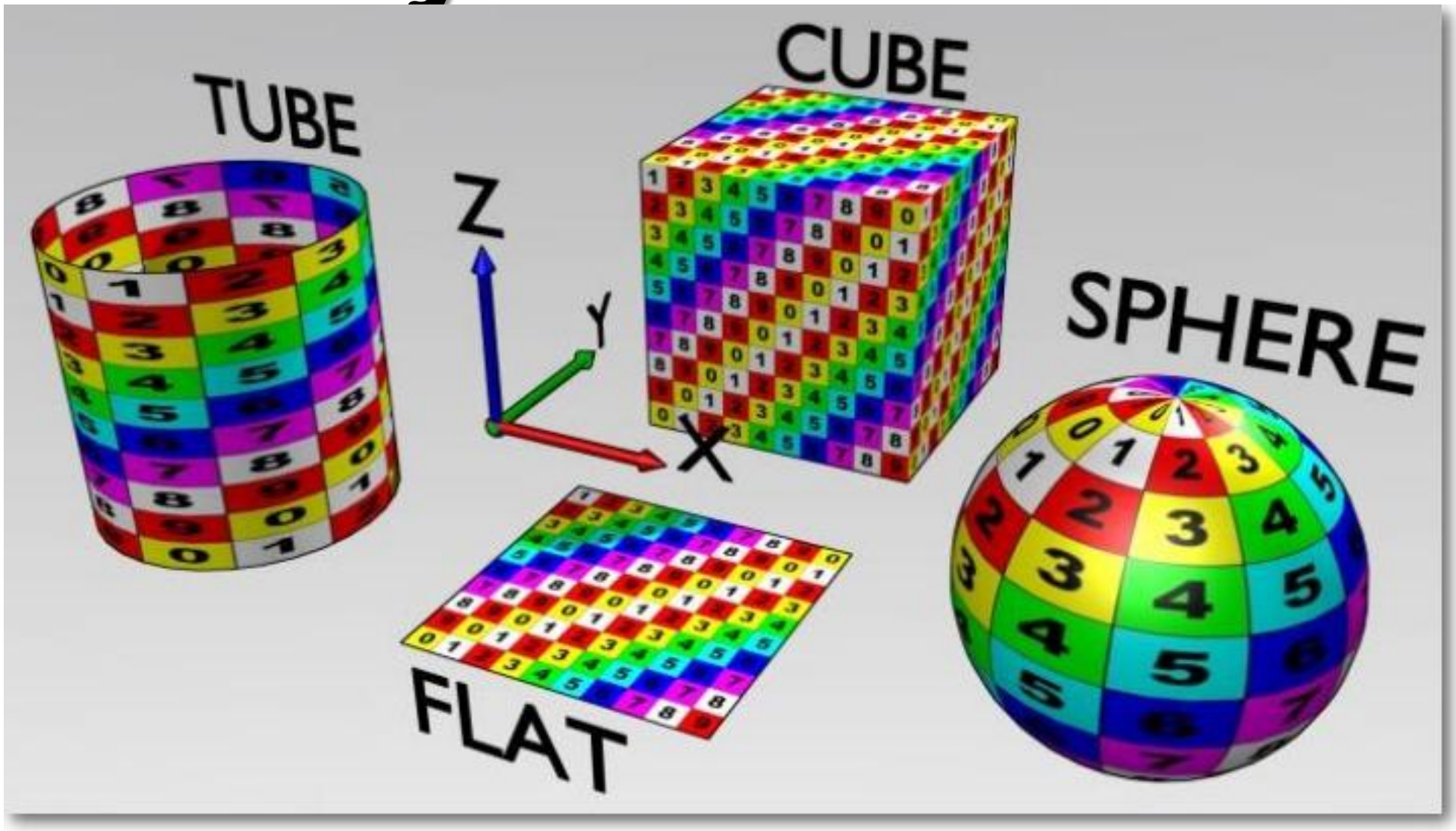
Parametric Surfaces

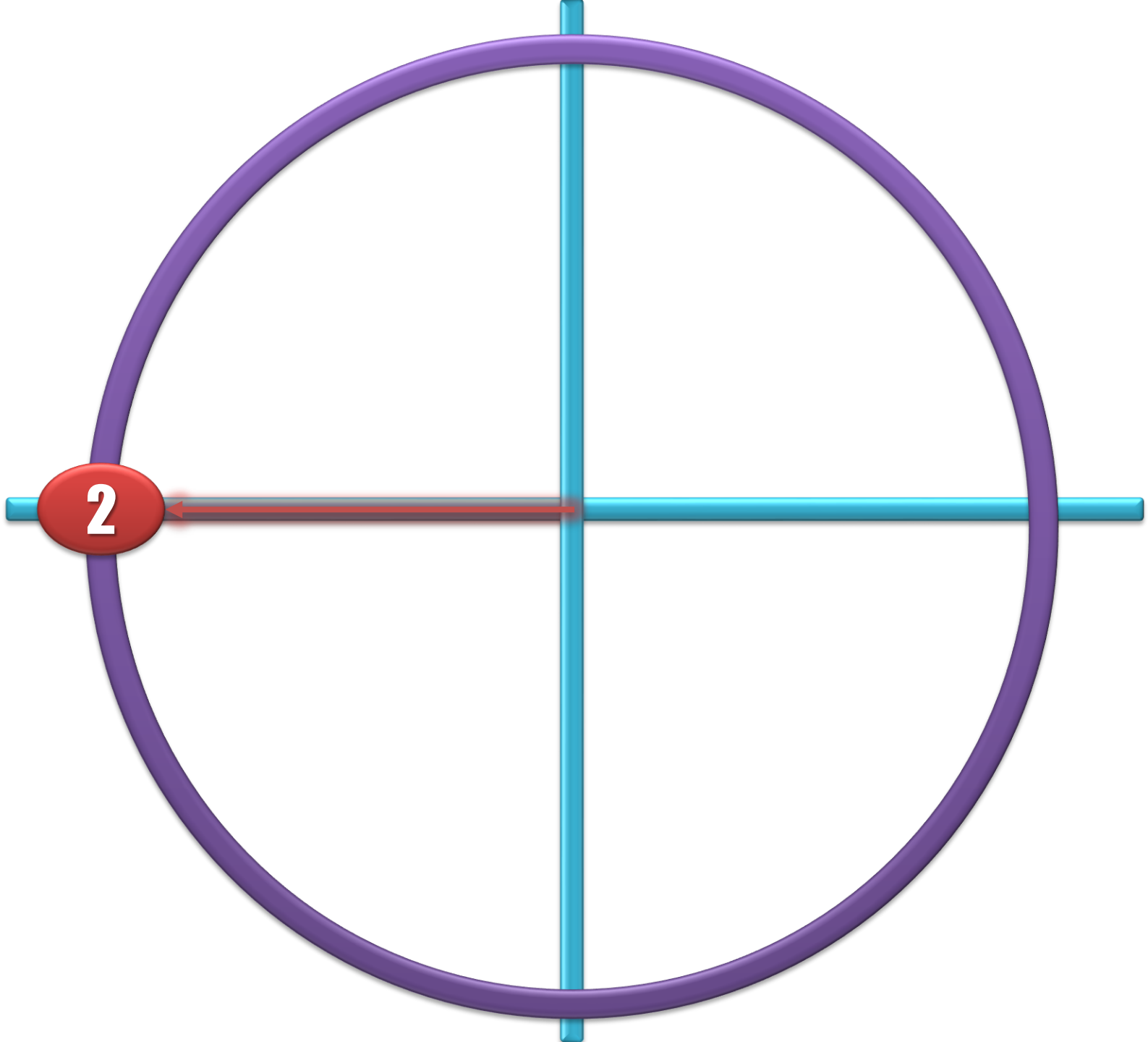
- Surface given by: $x = f(u, v)$, $y = g(u, v)$, $z = h(u, v)$





Summary:




2

Sampling

Sampling Texture

- A pixel maps to a non-rectangular region
- Perform map on center of pixels

Problems

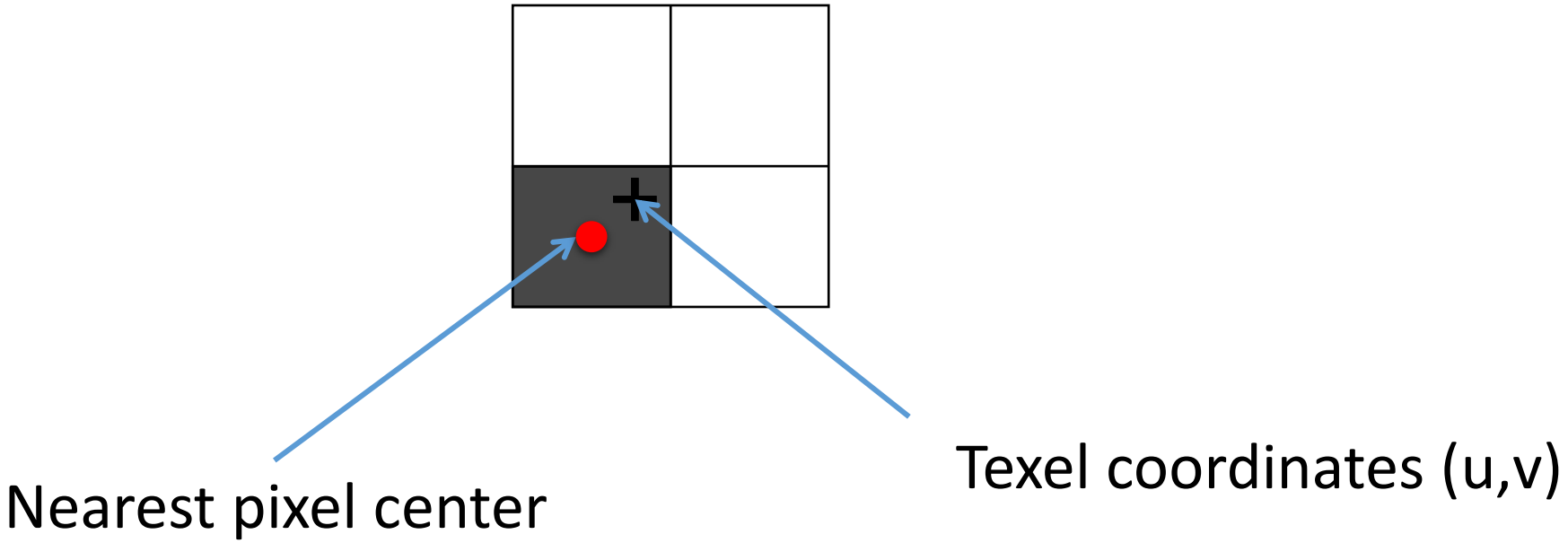
- Over-sampling the image:
  - same texel maps to several pixels
- Under-sampling the image:
  - pixels only sparsely sample the texels

Solutions

- Nearest neighbor sampling
- Bilinear sampling
- MipMapping

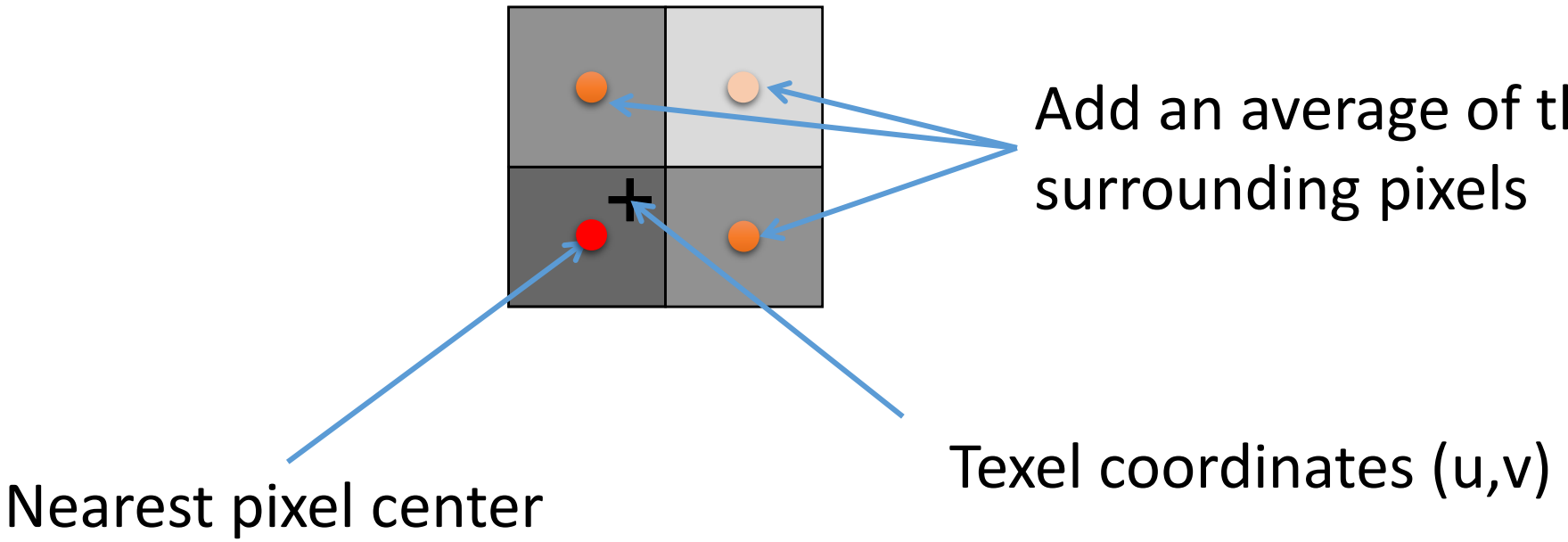
Nearest Neighbor

- Pick the pixel with the nearest center



Bilinear Interpolation

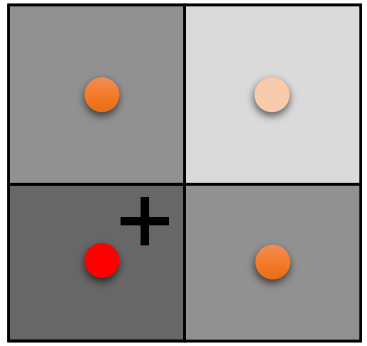
- Linearly interpolate across u and v

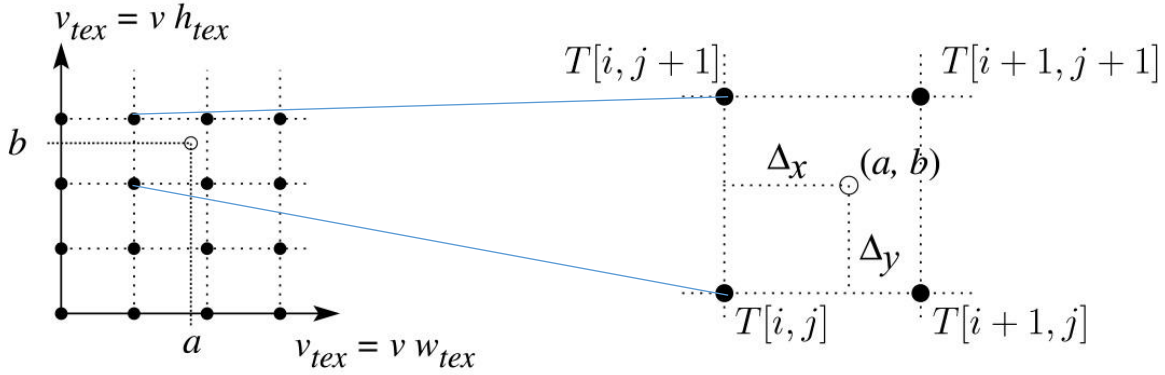


Bilinear Interpolation

- Linear interpolate in both directions

$$T(a,b) = T[i+\Delta_x, j+\Delta_y]$$
$$= (1-\Delta_x)(1-\Delta_y)T[i,j] + \Delta_x(1-\Delta_y)T[i+1,j]$$
$$+ (1-\Delta_x)\Delta_y T[i,j+1] + \Delta_x\Delta_y T[i+1,j+1]$$





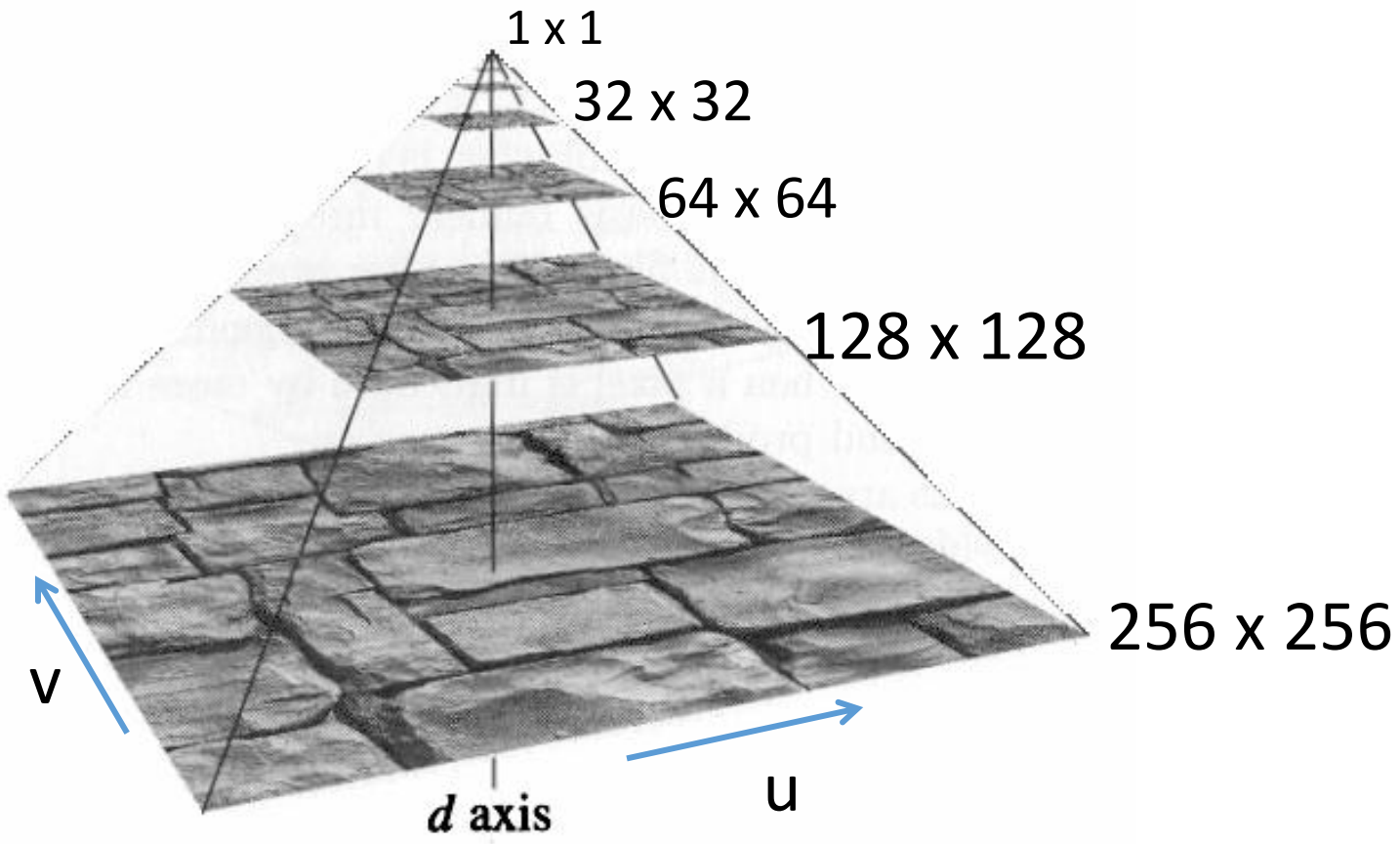
Mapping to texture pixel coords

Close-up

MipMaps

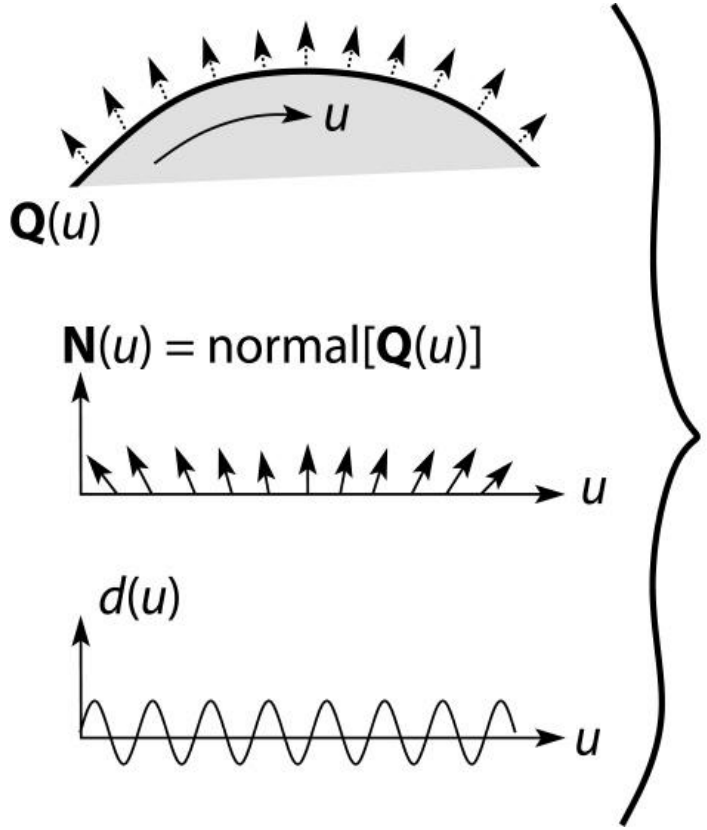
- Classically, texture sizes are a power of 2 (WebGL2 lifts this)
  - 64, 128, 256, 512, 1024
- Resample the image at lower resolutions
- Create a pyramid of texture images
- Interpolate the texture between two adjacent layers

MipMaps



Bump Mapping

- Perturb the Normal Vector

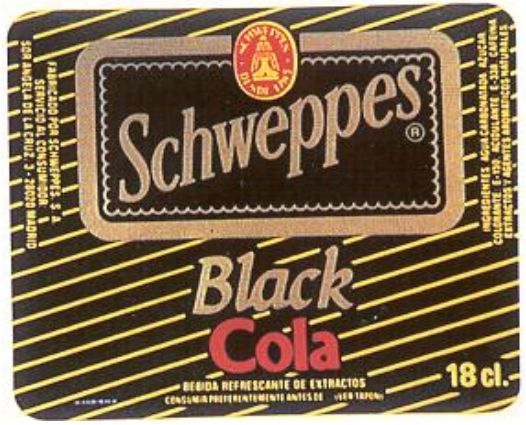


$$\tilde{\mathbf{Q}}(u) = \mathbf{Q}(u) + d(u)\mathbf{N}(u)$$

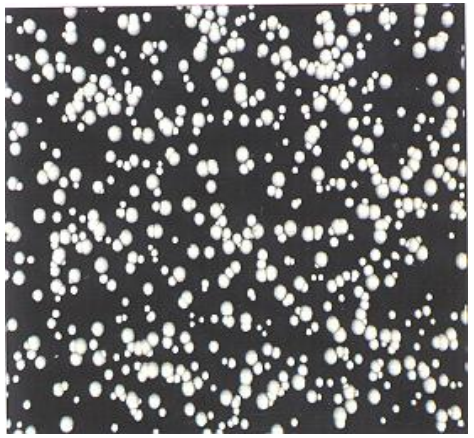


Bump Mapping

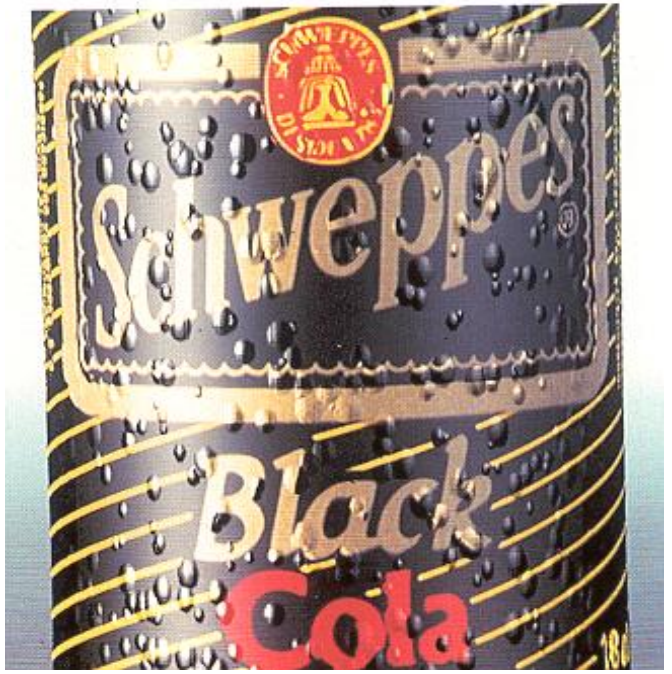
Texture 1



Texture 2
Bump Map

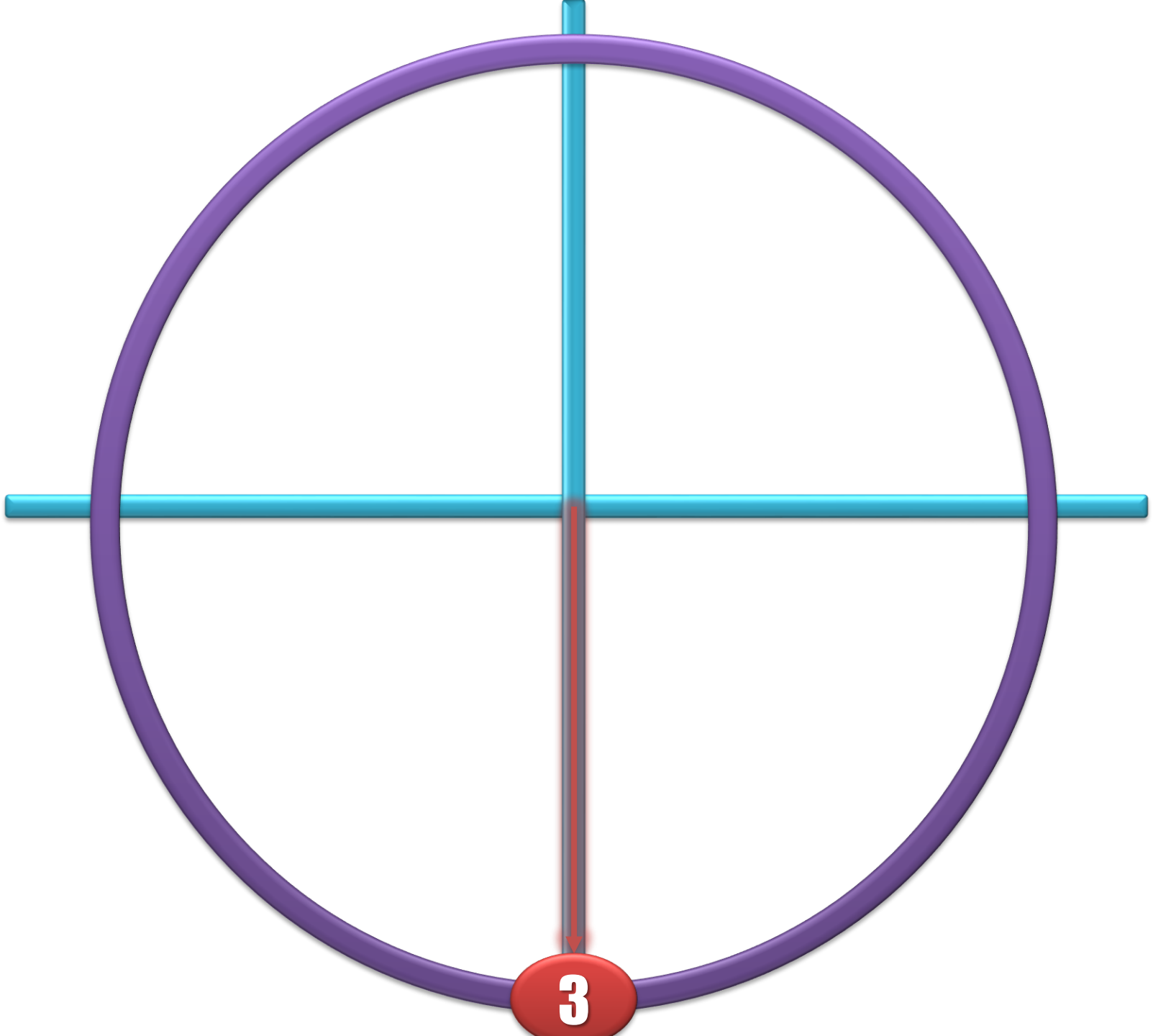


Rendered Image



Environment Mapping




3

WebGL Textures

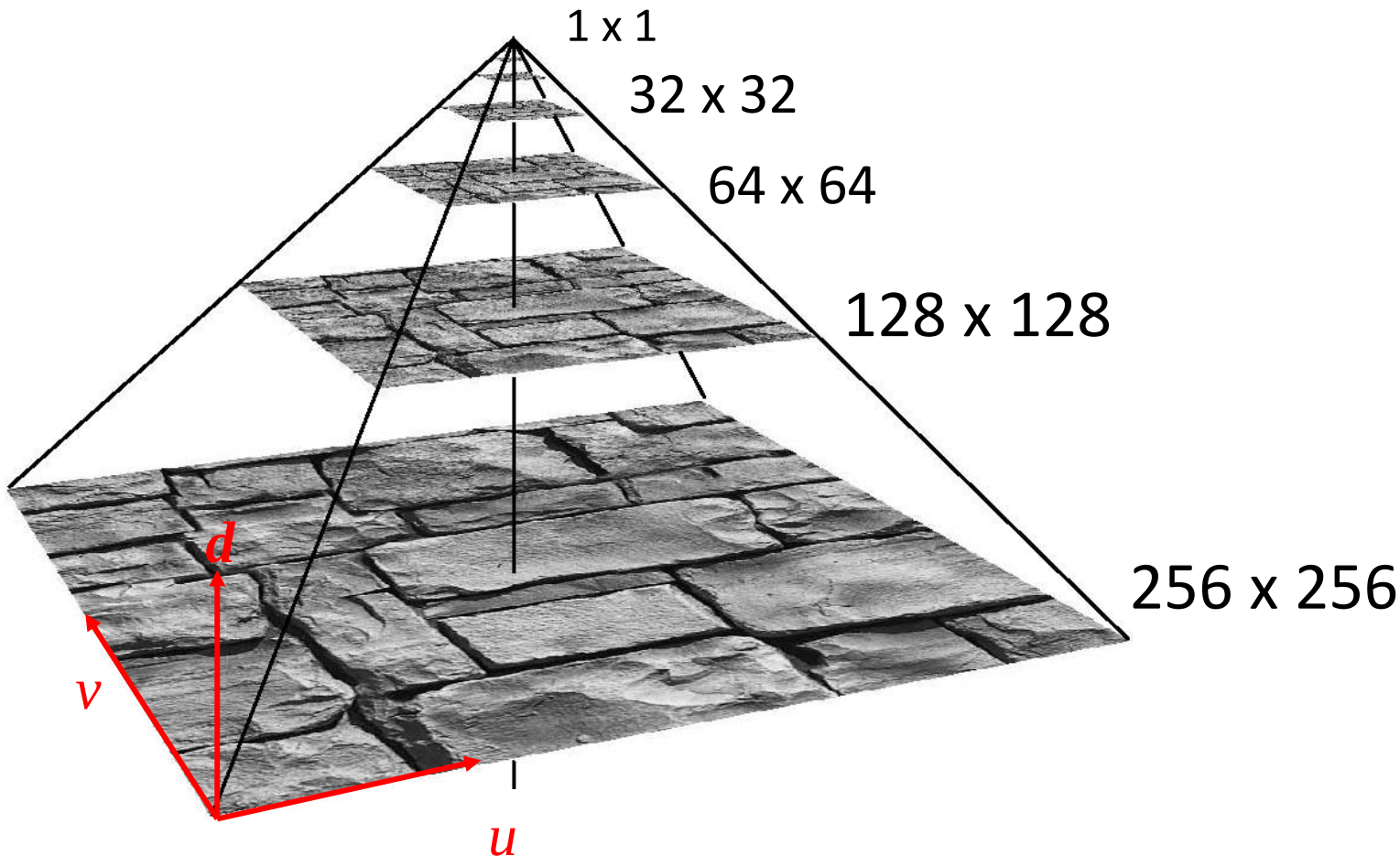
Read Texture Image

```
// Create a texture object; load the image into it
function initTextures() {
        const texture = gl.createTexture();
        const image = new Image();
        image.onload = function () {
                handleTextureLoaded(image, texture);
        };
        image.src = 'textureImage.png';
        return texture;
}
```

Bind Texture Buffers

```
// Callback function that is called after the texture image is loaded.
// Specifies that the new texture is the current texture by binding it to gl.TEXTURE_2D.
// Loaded image is passed into texImage2D() to write the image data into the texture.
function handleTextureLoaded(image, texture) {
        gl.bindTexture(gl.TEXTURE_2D, texture);
        gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA, gl.UNSIGNED_BYTE, image);
        gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);
        gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER,
                                                gl.LINEAR_MIPMAP_NEAREST);
        gl.generateMipmap(gl.TEXTURE_2D);
        gl.bindTexture(gl.TEXTURE_2D, null);
}
```

Power of 2 Image Sizes



Texture Image Sizes in WebGL2

- Any width and height works: 300 x 200 is as valid as 256 x 256
- Mipmaps and all wrap modes work at any size (WebGL1 could not)
- Devices differ in their largest texture: query gl.MAX_TEXTURE_SIZE
- Power-of-two sizes are no longer required; the WebGL1 rules are in the Appendix

Mipmaps and Wrapping at Any Size

- gl.generateMipmap() works for any image size
- gl.REPEAT and gl.MIRRORED_REPEAT work for any image size
- LINEAR_MIPMAP_LINEAR filters smoothly between mip levels
- Only WebGL1 needs the clamp-and-no-mipmap workaround (see Appendix)

Wrapping and Mipmaps: WebGL2 Code

```
gl.generateMipmap(gl.TEXTURE_2D);   // any image size


// smooth filtering between mip levels
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER,
gl.LINEAR_MIPMAP_LINEAR);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.LINEAR);


// let the texture repeat (tile) in s and t
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.REPEAT);
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.REPEAT);
```

Mapping Tex onto Faces

```
cubeVerticesTextureCoordBuffer = gl.createBuffer();
gl.bindBuffer(gl.ARRAY_BUFFER,
cubeVerticesTextureCoordBuffer);
var textureCoordinates = [
    // Front Face
    0.0,  0.0,
    1.0,  0.0,
    1.0,  1.0,
    0.0,  1.0,… ];
gl.bufferData(gl.ARRAY_BUFFER, new Float32Array
(textureCoordinates), gl.STATIC_DRAW);
```

Updating the VS

- First, we need the location of the attribute “aTextureCoord” in the initShaders() function:

```
textureCoordAttribute =
gl.getAttribLocation(shaderProgram, "aTextureCoord");
gl.enableVertexAttribArray(textureCoordAttribute);
```

Update the VS

```
<script id="shader-vs" type="x-shader/x-vertex">#version 300 es
    in vec3 aVertexPosition;
    in vec2 aTextureCoord;
    uniform mat4 uMVMatrix;
    uniform mat4 uPMatrix;
    out vec2 vTextureCoord; // communicates with the FS
    void main(void) {
      gl_Position = uPMatrix * uMVMatrix * vec4(aVertexPosition, 1.0);
      vTextureCoord = aTextureCoord;
    }
</script>
```

Update the FS

```
<script id="shader-fs" type="x-shader/x-fragment">#version 300 es
 precision highp float;
 in vec2 vTextureCoord;   // passed from VS
 uniform sampler2D uSampler;   // holds the texture image
 out vec4 outColor;
 void main(void) {
     outColor = texture(uSampler, vTextureCoord);
 }
</script>
```

FS Note

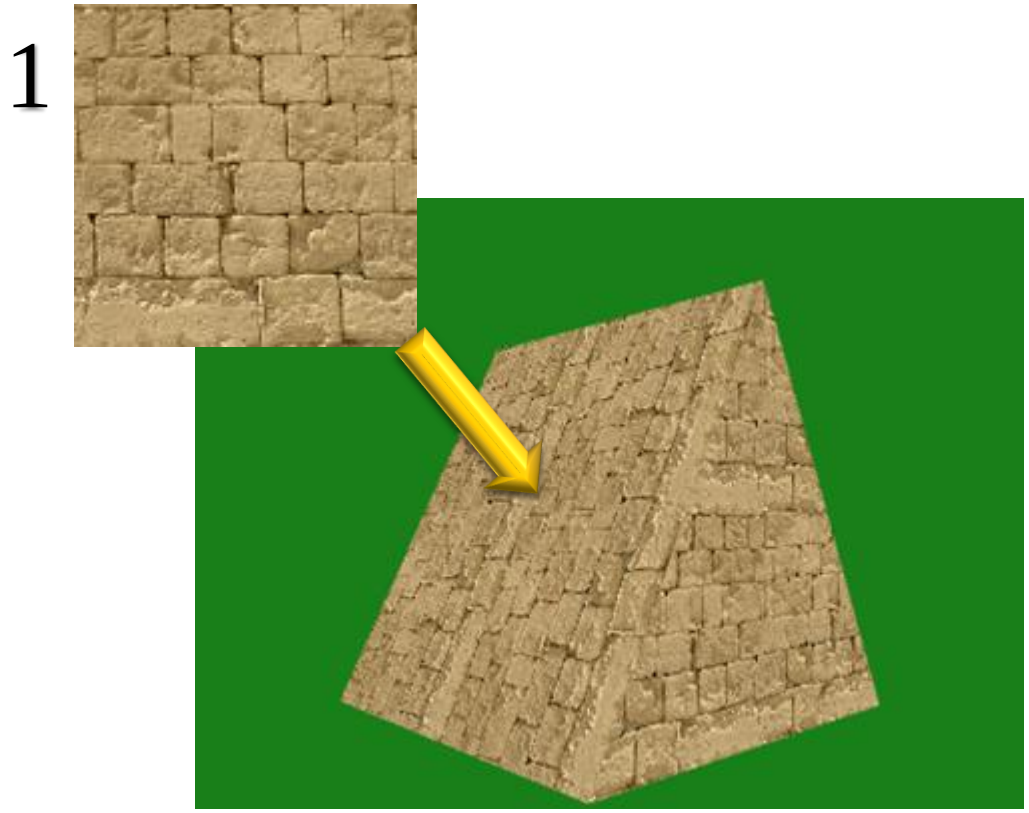
- Instead of assigning a color value to the fragment color, the fragment color is computed by fetching the **texel** (that is, the pixel within the texture) that the sampler says best maps to the fragment position.

Finally, drawing

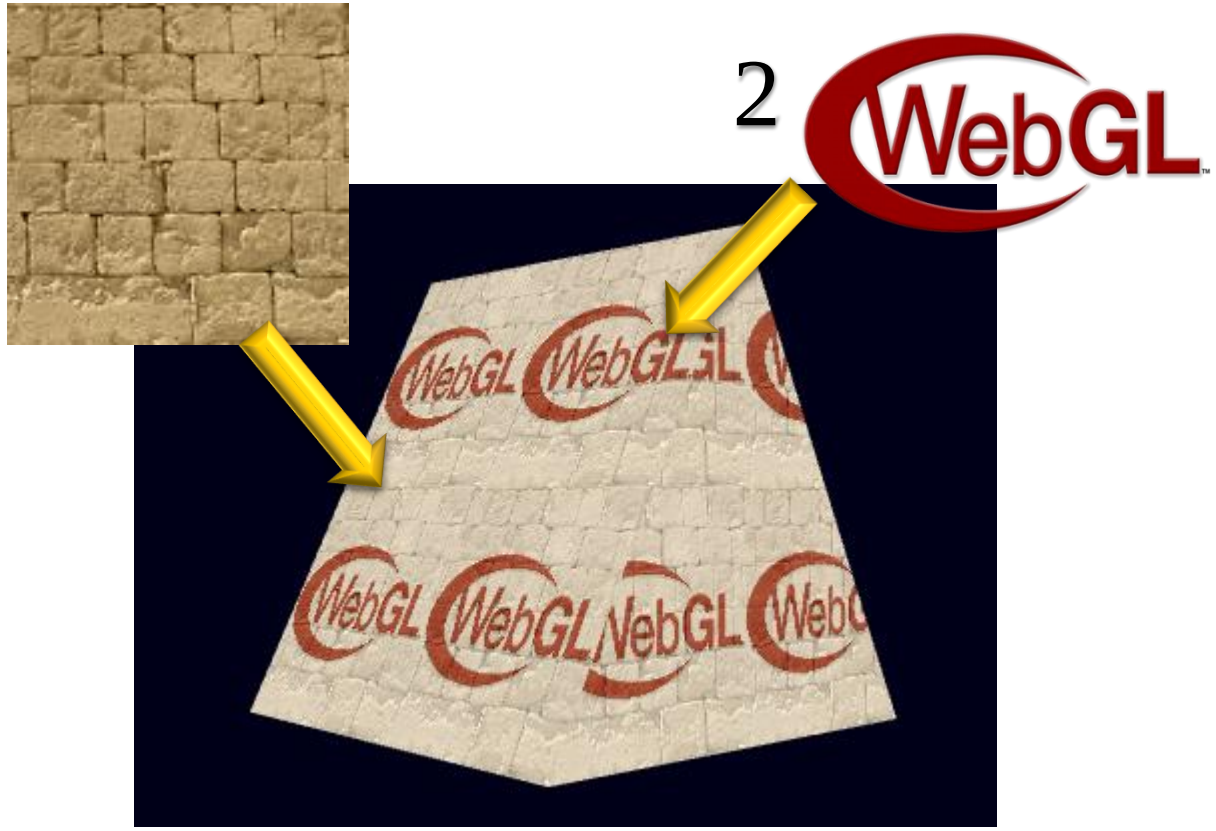
```
gl.activeTexture(gl.TEXTURE0);
gl.bindTexture(gl.TEXTURE_2D, cubeTexture);
gl.uniform1i(gl.getUniformLocation(
                    shaderProgram, "uSampler"), 0);
// note: GL provides 32 texture registers;
// the first of these is gl.TEXTURE0.
```

Multiple Textures

- Single Texture



- Double Texture



Multiple Texture

- You can use multi-textures to produce special effects such as:
  - light or height maps
  - bumpmapping (simulating bumps and wrinkles).
- We will just take the WebGL logo as a texture and mix it with the stone texture.

Two Textures

- We assign constant values to the variables:
  - STONE_TEXTURE
  - WEBGL_LOGO
- We will use them as our new array indices.
- Then, we change our texture variable declarations to be arrays
- Adjust the LoadTexture and SetupTexture functions to handle multiple textures.

Preparing for MultiTexture

```
// Global settings
const STONE_TEXTURE = 0,
      WEBGL_LOGO    = 1;
const texture = [], textureImage = [];
const textureFileName = [
        './texture/stone-128px.jpg',
        './texture/webgl-logo-512px.png'
];
```

Texture i = 0..1

```
function wglLoadTexture(i) {
        textureImage[i] = new Image();
        textureImage[i].onload = function () {
                wglSetupTexture(i);
        };
        textureImage[i].src = textureFileName[i];
}
```

Set up the uniform

```
glProgram.uDoTexturing =
gl.getUniformLocation(glProgram, "uDoTexturing");
gl.uniform1i(glProgram.uDoTexturing, 1);
```

Set up texture buffers

```
function wglSetupTexture(i) {
        gl.activeTexture(gl.TEXTURE0 + i);        // must specify the active texture
        texture[i] = gl.createTexture();
        gl.bindTexture(gl.TEXTURE_2D, texture[i]);
        gl.pixelStorei(gl.UNPACK_FLIP_Y_WEBGL, true);
        gl.texImage2D(gl.TEXTURE_2D, 0, gl.RGBA, gl.RGBA,
                gl.UNSIGNED_BYTE, textureImage[i]);
        gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MAG_FILTER, gl.NEAREST);
        gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.NEAREST);
        if( !gl.isTexture(texture[i]) ) {
                console.error("Error: Texture is invalid"); }
}
```

Get Sampler Locations

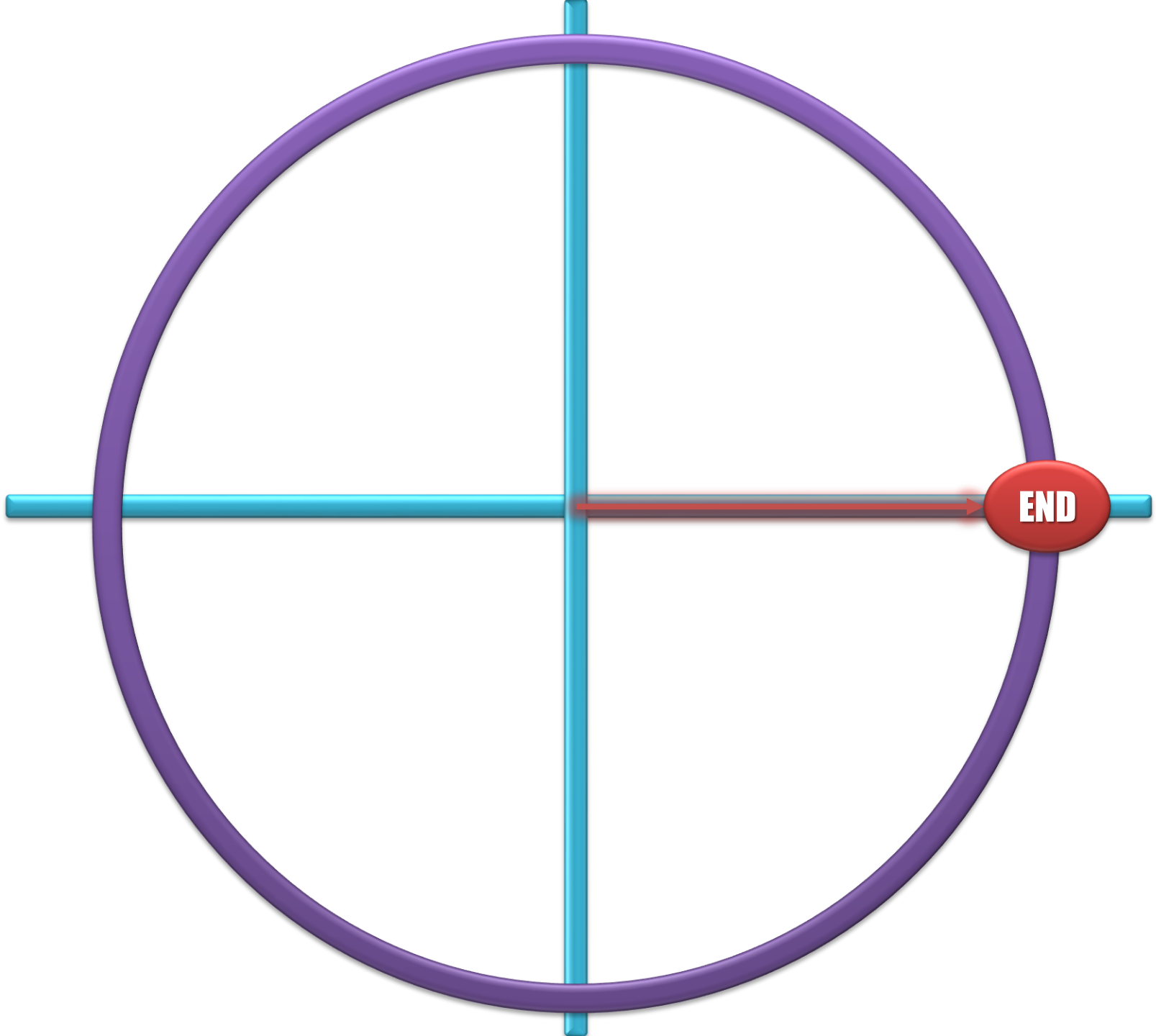
```
function getMatrixUniforms() {
        glProgram.pMatrixUniform   = gl.getUniformLocation(glProgram, "uPMatrix");
        glProgram.mvMatrixUniform  = gl.getUniformLocation(glProgram, "uMVMatrix");
        glProgram.samplerUniform   = gl.getUniformLocation(glProgram, "uSampler");
        glProgram.samplerUniform2  = gl.getUniformLocation(glProgram, "uSampler2");
        // each sampler reads from its own texture unit
        gl.uniform1i(glProgram.samplerUniform,  STONE_TEXTURE);
        gl.uniform1i(glProgram.samplerUniform2, WEBGL_LOGO);
}
```

Finally, the Frag. Shader

```
<script id="shader-fs" type="x-shader/x-fragment">#version 300 es
        precision highp float;
        in vec2 vTextureCoord;
        uniform sampler2D uSampler;
        uniform sampler2D uSampler2;
        uniform int uDoTexturing;
        out vec4 outColor;
        // ...
```

Fragment Shader

```
void main(void) {
        if (uDoTexturing == 1) {        // check for texturing flag
                vec4 stoneColor     = texture(uSampler,  vTextureCoord);
                vec4 webglLogoColor = texture(uSampler2, vTextureCoord);
                outColor = mix(stoneColor, webglLogoColor, 0.5);
        } else {
                outColor = vec4(1.0, 0.1, 0.1, 1.0);
        }
}
</script>
```


END

Appendix

Original WebGL1 / GLSL ES 1.00 code slides, kept for reference. The lecture itself uses WebGL2 and GLSL ES 3.00.

Read Texture Image (WebGL1)

Old version P.44

```
// Create a GL texture object “cubeTexture”
// Load the texture from the image file
function initTextures() {
        cubeTexture = gl.createTexture();
        cubeImage = new Image();
        cubeImage.onload = function() {
                handleTextureLoaded(cubeImage, cubeTexture);
         } cubeImage.src = “textureImage.png";
}
```

NPOT: Not Power-of-Two (WebGL1)

Old version P.47

- Texture images with sides lengths (x,y)
  - power of two are ideal: e.g. 1, 2, 4, 8, 16, 32,
- Not all devices can support >4096.
- But, if the images are from outside, they may not be 2^n.
- How to solve this problem?

NPOT Solutions (WebGL1)

- Best solution:
  - Reduce the image into a square image with sides as a 2^n
- Second best solution:
  - Could make the texture images the same as the native resolution of the monitor

NPOT Problems (WebGL1)

Old version P.49

- Cannot use MIPMAPPING
- Cannot REPEAT, tile or wrap
- Mipmapping and UV repeat can be disabled with gl.texParameteri()

NPOT disabling MipMaps (WebGL1)

Old version P.50

```
// gl.NEAREST also allowed, instead of gl.LINEAR, as neither mipmap.
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_MIN_FILTER, gl.LINEAR);

// Prevents s-coordinate wrapping (repeating).
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_S, gl.CLAMP_TO_EDGE);

// Prevents t-coordinate wrapping (repeating).
gl.texParameteri(gl.TEXTURE_2D, gl.TEXTURE_WRAP_T, gl.CLAMP_TO_EDGE);
```

Old version P.53

Update the VS (WebGL1)

```
<script id="shader-vs" type="x-shader/x-vertex">
    attribute vec3 aVertexPosition;
    attribute vec2 aTextureCoord;
    uniform mat4 uMVMatrix;
    uniform mat4 uPMatrix;
    varying highp vec2 vTextureCoord; // communicates with the FS
    void main(void) {
      gl_Position = uPMatrix * uMVMatrix * vec4(aVertexPosition, 1.0);
      vTextureCoord = aTextureCoord;
    }
</script>
```

Old version P.54

Update the FS (WebGL1)

```
<script id="shader-fs" type="x-shader/x-fragment">
  varying highp vec2 vTextureCoord;  // passed from VS
  uniform sampler2D uSampler; // holds the texture image
  void main(void) {
        gl_FragColor = texture2D(uSampler,
                    vec2(vTextureCoord.s, vTextureCoord.t));
  }
</script>
```

Preparing for MultiTexture (WebGL 1)

```
// Global settings:
STONE_TEXTURE                   = 0,
WEBGL_LOGO                      = 1,
texture                         = [],
textureImage                    = [];
textureFileName[]               = [
                    "./texture/stone-128px.jpg“,
                    “./texture/webgl-logo-512px.png”
                    ];
```

Texture i = 0..1 (WebGL1)

```
function wglLoadTexture(i)
{       textureImage[i] = new Image();
        textureImage[i].onload = function() {
                setupTexture(i);
                gl.uniform1i(glProgram.samplerUniform, i);
        }
        textureImage[i].src = textureFileName[i];
}
```

Get Sampler Locations (WebGL1)

```
function getMatrixUniforms(){
        glProgram.pMatrixUniform =
        gl.getUniformLocation(glProgram, "uPMatrix");
        glProgram.mvMatrixUniform =
        gl.getUniformLocation(glProgram, "uMVMatrix");
        glProgram.samplerUniform =
        gl.getUniformLocation(glProgram, "uSampler");
        glProgram.samplerUniform2 =
        gl.getUniformLocation(glProgram, "uSampler2");
}
```

Finally, the Frag. Shader (WebGL1)

```
<script id="shader-fs" type="x-shader/x-fragment">
        varying highp vec2 vTextureCoord;
        uniform sampler2D uSampler;
        uniform sampler2D uSampler2;
        uniform int uDoTexturing;
// ...
```

Fragment Shader (WebGL1)

```
void main(void) {
        if(uDoTexturing == 1){ // check for texturing flag
                highp vec4 stoneColor = texture2D(uSampler,
                                                  vec2(vTextureCoord.st));
                highp vec4 webglLogoColor = texture2D(uSampler2,
                                                  vec2(vTextureCoord.st));
                gl_FragColor = mix(stoneColor, webglLogoColor, 0.5);
        }else{
                gl_FragColor = vec4(1.0, 0.1, 0.1, 1.0);
        }
}
</script>
```